STRINGS

Trim leading and trailing white-space from string

This is an alternative to sed, awk, perl and other tools. The function below works by finding all leading and trailing white-space and removing it from the start and end of the string. The `:` built-in is used in place of a temporary variable.

Example Function:

```
trim_string() {
    # Usage: trim_string "   example   string    "
    : "${1#"${1%%[![:space:]]*}"}"
    : "${_%"${_##*[![:space:]]}"}"
    printf '%s\n' "$_"
}
```

Example Usage:

```
$ trim_string "    Hello,  World    "
Hello,  World

$ name="   John Black  "
$ trim_string "$name"
John Black
```

Trim all white-space from string and truncate spaces

This is an alternative to sed, awk, perl and other tools. The function below works by abusing word splitting to create a new string without leading/trailing white-space and with truncated spaces.

Example Function:

```
# shellcheck disable=SC2086,SC2048
trim_all() {
    # Usage: trim_all "   example   string    "
    set -f
    set -- $*
    printf '%s\n' "$*"
    set +f
}
```

Example Usage:

```
$ trim_all "    Hello,    World    "
Hello, World

$ name="   John   Black  is     my    name.    "
$ trim_all "$name"
John Black is my name.
```

Use regex on a string

The result of bash's regex matching can be used to replace sed for a large number of use-cases.

CAVEAT: This is one of the few platform dependent bash features. bash will use whatever regex engine is installed on the user's system. Stick to POSIX regex features if aiming for compatibility.

CAVEAT: This example only prints the first matching group. When using multiple capture groups some modification is needed.

Example Function:

```
regex() {
    # Usage: regex "string" "regex"
    [[ $1 =~ $2 ]] && printf '%s\n' "${BASH_REMATCH[1]}"
}
```

Example Usage:

```
$ # Trim leading white-space.
$ regex '    hello' '^\s*(.*)'
hello

$ # Validate a hex color.
$ regex "#FFFFFF" '^(#?([a-fA-F0-9]{6}|[a-fA-F0-9]{3}))$'
#FFFFFF

$ # Validate a hex color (invalid).
$ regex "red" '^(#?([a-fA-F0-9]{6}|[a-fA-F0-9]{3}))$'
# no output (invalid)
```

Example Usage in script:

```
is_hex_color() {
    if [[ $1 =~ ^(#?([a-fA-F0-9]{6}|[a-fA-F0-9]{3}))$ ]]; then
        printf '%s\n' "${BASH_REMATCH[1]}"
    else
        printf '%s\n' "error: $1 is an invalid color."
        return 1
    fi
}

read -r color
is_hex_color "$color" || color="#FFFFFF"

# Do stuff.
```

Split a string on a delimiter

CAVEAT: Requires bash 4+

This is an alternative to `cut`, `awk` and other tools.

Example Function:

```
split() {
   # Usage: split "string" "delimiter"
   IFS=$'\n' read -d "" -ra arr <<< "${1//$2/$'\n'}"
   printf '%s\n' "${arr[@]}"
}
```

Example Usage:

```
$ split "apples,oranges,pears,grapes" ","
apples
oranges
pears
grapes

$ split "1, 2, 3, 4, 5" ", "
1
2
3
4
5

# Multi char delimiters work too!
$ split "hello---world---my---name---is---john" "---"
hello
world
my
name
is
john
```

Change a string to lowercase

CAVEAT: Requires bash 4+

Example Function:

```
lower() {
    # Usage: lower "string"
    printf '%s\n' "${1,,}"
}
```

Example Usage:

```
$ lower "HELLO"
hello

$ lower "HeLlO"
hello

$ lower "hello"
```

```
hello
```

Change a string to uppercase

CAVEAT: Requires bash 4+

Example Function:

```
upper() {
    # Usage: upper "string"
    printf '%s\n' "${1^^}"
}
```

Example Usage:

```
$ upper "hello"
HELLO

$ upper "HeLlO"
HELLO

$ upper "HELLO"
HELLO
```

Reverse a string case

CAVEAT: Requires bash 4+

Example Function:

```
reverse_case() {
    # Usage: reverse_case "string"
    printf '%s\n' "${1~~}"
}
```

Example Usage:

```
$ reverse_case "hello"
HELLO

$ reverse_case "HeLlO"
hElLo

$ reverse_case "HELLO"
hello
```

Trim quotes from a string

Example Function:

```
trim_quotes() {
    # Usage: trim_quotes "string"
    : "${1//\'}"
```

```
    printf '%s\n' "${_//\"}"
}
```

Example Usage:

```
$ var="'Hello', \"World\""
$ trim_quotes "$var"
Hello, World
```

Strip all instances of pattern from string

Example Function:

```
strip_all() {
    # Usage: strip_all "string" "pattern"
    printf '%s\n' "${1//$2}"
}
```

Example Usage:

```
$ strip_all "The Quick Brown Fox" "[aeiou]"
Th Qck Brwn Fx

$ strip_all "The Quick Brown Fox" "[[:space:]]"
TheQuickBrownFox

$ strip_all "The Quick Brown Fox" "Quick "
The Brown Fox
```

Strip first occurrence of pattern from string

Example Function:

```
strip() {
    # Usage: strip "string" "pattern"
    printf '%s\n' "${1/$2}"
}
```

Example Usage:

```
$ strip "The Quick Brown Fox" "[aeiou]"
Th Quick Brown Fox

$ strip "The Quick Brown Fox" "[[:space:]]"
TheQuick Brown Fox
```

Strip pattern from start of string

Example Function:

```
lstrip() {
    # Usage: lstrip "string" "pattern"
```

```
    printf '%s\n' "${1##$2}"
}
```

Example Usage:

```
$ lstrip "The Quick Brown Fox" "The "
Quick Brown Fox
```

Strip pattern from end of string

Example Function:

```
rstrip() {
    # Usage: rstrip "string" "pattern"
    printf '%s\n' "${1%%$2}"
}
```

Example Usage:

```
$ rstrip "The Quick Brown Fox" " Fox"
The Quick Brown
```

Percent-encode a string

Example Function:

```
urlencode() {
    # Usage: urlencode "string"
    local LC_ALL=C
    for (( i = 0; i < ${#1}; i++ )); do
        : "${1:i:1}"
        case "$_" in
            [a-zA-Z0-9.~_-])
                printf '%s' "$_"
            ;;

            *)
                printf '%%%02X' "'$_"
            ;;
        esac
    done
    printf '\n'
}
```

Example Usage:

```
$ urlencode "https://github.com/dylanaraps/pure-bash-bible"
https%3A%2F%2Fgithub.com%2Fdylanaraps%2Fpure-bash-bible
```

Decode a percent-encoded string

Example Function:

```
urldecode() {
    # Usage: urldecode "string"
    : "${1//+/ }"
    printf '%b\n' "${_//%/\\x}"
}
```

Example Usage:

```
$ urldecode "https%3A%2F%2Fgithub.com%2Fdylanaraps%2Fpure-bash-bible"
https://github.com/dylanaraps/pure-bash-bible
```

Check if string contains a sub-string

Using a test:

```
if [[ $var == *sub_string* ]]; then
    printf '%s\n' "sub_string is in var."
fi

# Inverse (substring not in string).
if [[ $var != *sub_string* ]]; then
    printf '%s\n' "sub_string is not in var."
fi

# This works for arrays too!
if [[ ${arr[*]} == *sub_string* ]]; then
    printf '%s\n' "sub_string is in array."
fi
```

Using a case statement:

```
case "$var" in
    *sub_string*)
        # Do stuff
    ;;

    *sub_string2*)
        # Do more stuff
    ;;

    *)
        # Else
    ;;
esac
```

Check if string starts with sub-string

```
if [[ $var == sub_string* ]]; then
    printf '%s\n' "var starts with sub_string."
fi
```

```
# Inverse (var does not start with sub_string).
if [[ $var != sub_string* ]]; then
    printf '%s\n' "var does not start with sub_string."
fi
```

Check if string ends with sub-string

```
if [[ $var == *sub_string ]]; then
    printf '%s\n' "var ends with sub_string."
fi

# Inverse (var does not end with sub_string).
if [[ $var != *sub_string ]]; then
    printf '%s\n' "var does not end with sub_string."
fi
```